

Week 8 Assignment: Serialization for the Parking System Application:

Client/Server

Master of Science

Information Technology

Object-Oriented Mthd & Pgm II

Gabriel Twizerimana

University of Denver College of Professional Studies

May 24, 2026

Faculty: Nathan Braun, MS

Dean: Bobbie Kite, PhD

Table of Contents

Assignment- Serialization for the Parking System Application: Client/Server	1
Introduction	1
Architectural Layout and Protocol Design	2
The Polymorphic Request Wrapper.....	2
Technical Challenges and Reflections	3
Managing Stateful Context in Distributed Streams	3
Reflection Constraints vs. Immutability	4
Isolation of Network Test Environments	5
Conclusion	6
Appendix.....	7
Server Side.....	7
Client Side.....	7
References	8

Assignment- Serialization for the Parking System

Application: Client/Server

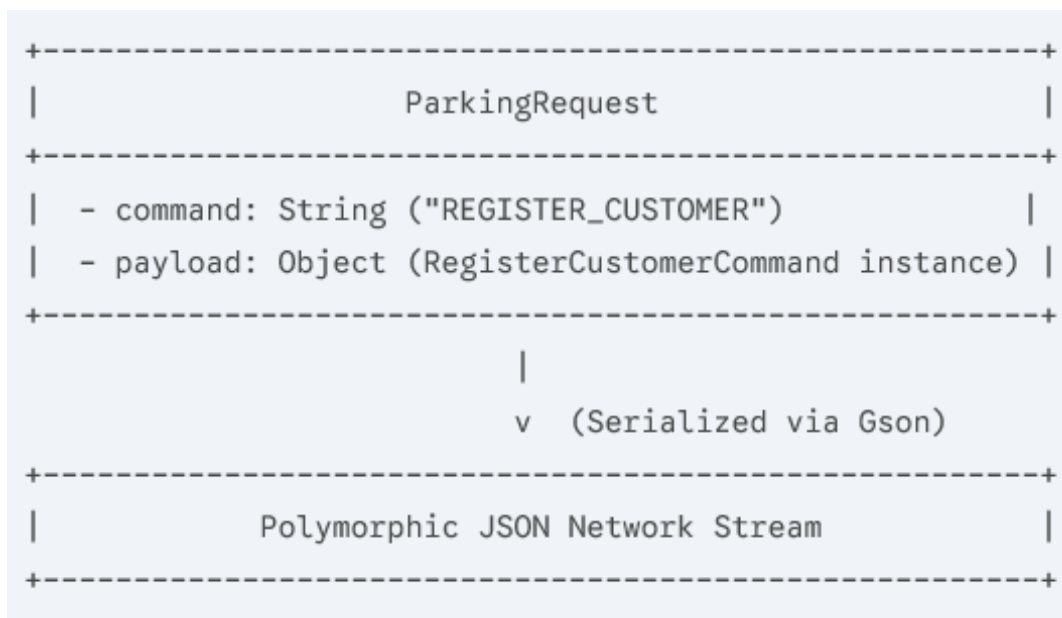
Introduction

In the development of network-centric, distributed software applications, establishing a reliable data-interchange agreement between remote client terminals and a central processing hub is critical. Traditional architectural patterns often separate data transmission components from the business rules that process them. However, as systems grow in complexity, this separation can lead to duplicate validation layers, bloated codebases, and structural rigidity.

This paper outlines the technical design of an alternative strategy: command-driven distributed architecture. By converting the operational commands themselves, specifically `RegisterCustomerCommand` and `RegisterCarCommand` into the primary data transport mechanisms over a raw TCP socket layer, we eliminated intermediate Data Transfer Objects (DTOs). The following sections detail this architectural layout and reflect on the significant design challenges faced during implementation, specifically focusing on handling stateful server-side context references during network serialization and managing object-oriented dependencies without external mocking engines.

Architectural Layout and Protocol Design

The underlying architecture is built on a streamlined Command Pattern wrapped within a polymorphic network envelope. Instead of generating distinct messaging payloads for every domain entity transaction (e.g., a standalone CustomerPayload or CarPayload), the system treats the executive command as the data container itself.



The Polymorphic Request Wrapper

The entry point for network transmission is the ParkingRequest class, an envelope container consisting of two fields:

- **command (String):** A routing tag analyzed by the server to determine the transaction context.
- **payload (Object):** A widening-type field that holds the command instance.

When an operation is initiated, the application populates the command fields with primitive values (id, name, licensePlate) and drops the command instance directly into the ParkingRequest container. The entire object graph is marshaled into a single JSON line through Google's Gson library and transmitted over an open TCP socket.

On the receiving end, the server runs an accept-polling loop. When a packet arrives, the server performs a two-pass extraction:

1. It parses the top-level string to read the command routing tag.
2. It deserializes the nested payload directly into the matching concrete class (RegisterCustomerCommand or RegisterCarCommand).

Once the command is reconstructed, the server invokes its standard. execute(null) entry point, processes the business rules against the underlying domain layer, and returns a uniform ParkingResponse packet containing an integer status signaling code and a descriptive confirmation string.

Technical Challenges and Reflections

Shifting from a passive DTO architecture to a dynamic, command-driven serialization framework exposed hidden rigidities in our system design. Overcoming these compilation and runtime hurdles required structural refactoring.

Managing Stateful Context in Distributed Streams

The primary hurdle encountered during this refactoring was the presence of the ParkingOffice office reference inside the command structures. To execute business logic natively, commands need direct access to this stateful server-side manager. However, a ParkingOffice contains active collections, registry memory, and observer lists that cannot cross a network stream. Attempting to serialize a live manager object results in runtime crashes, infinite serialization loops, or massive data leaks across the wire.

To resolve this boundary conflict, the server-side reference was decoupled from the serialization path by marking the office field with the Java transient keyword:

Field Isolation: private transient ParkingOffice office;

The transient modifier tells Gson to completely ignore the field during serialization. While this protected the network boundary, it created a secondary problem: upon deserialization on the server, the office field arrived as null.

To bridge this gap, an explicit dependency injection hook was engineered into the commands via a `setOffice(ParkingOffice office)` method. When the server reconstructs a command from the network stream, it intercepts the object before execution, passes its local active `ParkingOffice` instance into the command, and re-hydrates the stateful execution context dynamically.

Reflection Constraints vs. Immutability

A secondary challenge emerged concerning Java's strict object-oriented access rules. The initial command definitions used final fields to enforce immutability after construction. However, reflection-based deserialization libraries like Gson bypass standard multi-argument constructors. Instead, they require a stable, explicit zero-argument constructor to create an empty shell instance before using reflection to populate private fields directly from the JSON token key-value pairs.

Declaring blank constructors alongside final field properties creates an invalid compilation state in Java. To maintain compatibility with reflection frameworks, the final constraints on data variables (`id`, `name`, `licensePlate`, `carType`) were removed, and explicit no-argument constructors were added. This adjustment allows the reflection engine to instantiate the commands cleanly at runtime, ensuring smooth data binding across socket connections.

Isolation of Network Test Environments

The final technical challenge involved writing comprehensive, automated integration tests without using third-party mocking frameworks (such as Mockito). Validating the server's input stream reader and the client's output stream writer required spinning up live network sockets. This introduced the risk of port collision errors (`BindException`) during simultaneous automated Maven build runs.

[Maven Build Lifecycle]

|

|—► `ServerClientTest` —► Active Port: 12346 —► Validates Stream Output

|

└─► `RegisterCarCommandTest` —► Native Validation —► Validates Domain Mutation

This dependency issue was resolved by implementing parameter-driven port allocation. The production environment was hardcoded to default port 12345, while individual integration tests, such as `ServerClientTest`, were assigned isolated out-of-band ports (e.g., 12346).

To simulate a remote server host without external dependencies, a raw programmatic server socket was built inside the test framework's `@BeforeEach` setup cycle, running on a separate background execution thread. This background thread captured the outbound JSON packet, verified its structural formatting, and flushed a mock JSON string response back to the client. This approach allowed the entire network infrastructure to be tested cleanly in isolation, with all sockets safely closed via an explicit `@AfterEach` teardown sequence, ensuring zero environment cross-talk.

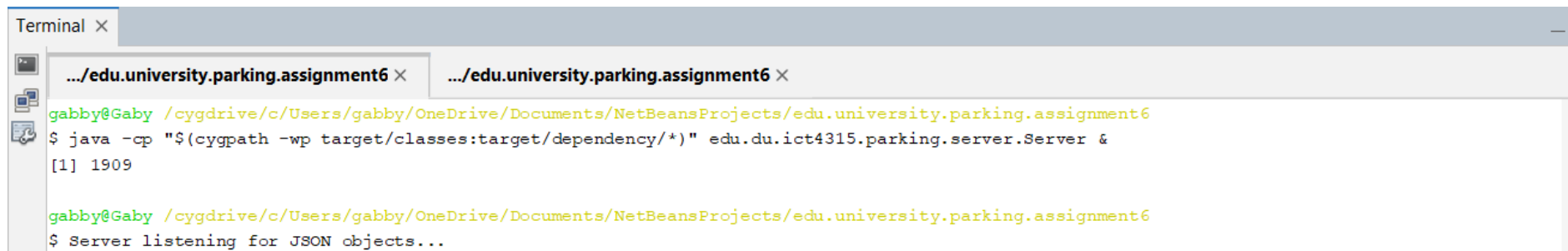
Conclusion

Refactoring the distributed network layer to use a command-driven architecture simplified the codebase by turning operational commands into self-contained data carriers. Eliminating standalone payload classes minimized structural redundancy and concentrated transactional logic within the command definitions.

Forcing the system to be tested using only core, native Java utilities, without the abstraction of mock frameworks, exposed hidden tight couplings early in the development lifecycle. The resulting design achieves a clean separation of concerns: it protects internal server states using transient network boundaries, ensures compatibility with reflection frameworks, and provides a stable, decoupled foundation for high-performance network communication.

Appendix

Server Side

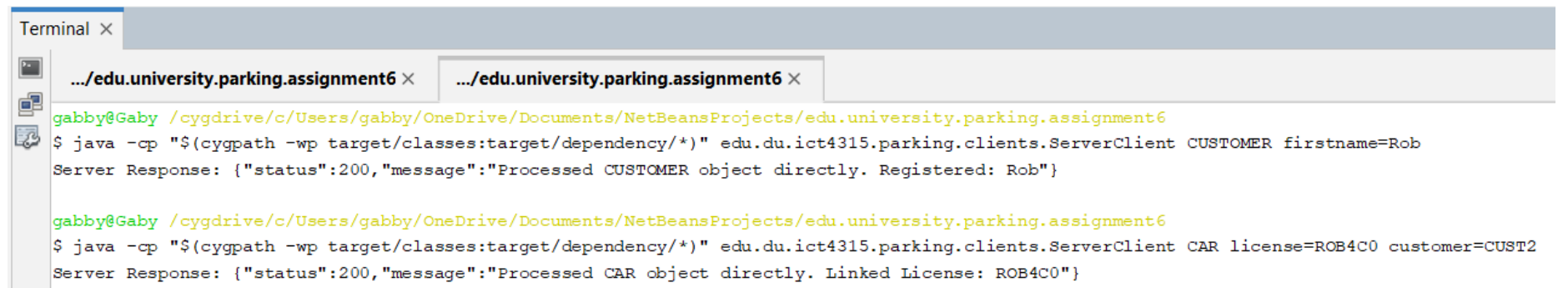


A terminal window titled "Terminal" with two tabs, both labeled ".../edu.university.parking.assignment6". The terminal shows the following commands and output:

```
gabby@Gaby /cygdrive/c/Users/gabby/OneDrive/Documents/NetBeansProjects/edu.university.parking.assignment6
$ java -cp "$(cygpath -wp target/classes:target/dependency/*)" edu.du.ict4315.parking.server.Server &
[1] 1909

gabby@Gaby /cygdrive/c/Users/gabby/OneDrive/Documents/NetBeansProjects/edu.university.parking.assignment6
$ Server listening for JSON objects...
```

Client Side



A terminal window titled "Terminal" with two tabs, both labeled ".../edu.university.parking.assignment6". The terminal shows the following commands and output:

```
gabby@Gaby /cygdrive/c/Users/gabby/OneDrive/Documents/NetBeansProjects/edu.university.parking.assignment6
$ java -cp "$(cygpath -wp target/classes:target/dependency/*)" edu.du.ict4315.parking.clients.ServerClient CUSTOMER firstname=Rob
Server Response: {"status":200,"message":"Processed CUSTOMER object directly. Registered: Rob"}

gabby@Gaby /cygdrive/c/Users/gabby/OneDrive/Documents/NetBeansProjects/edu.university.parking.assignment6
$ java -cp "$(cygpath -wp target/classes:target/dependency/*)" edu.du.ict4315.parking.clients.ServerClient CAR license=ROB4C0 customer=CUST2
Server Response: {"status":200,"message":"Processed CAR object directly. Linked License: ROB4C0"}
```

References

Goetz, Brian, Tim Peierls, Joshua Bloch, Joseph Bowbeer, David Holmes, and Doug Lea. 2006. *Java Concurrency in Practice*. Upper Saddle River, NJ: Addison-Wesley.

Harold, Elliotte Rusty. 2013. *Java Network Programming*. 4th ed. Sebastopol, CA: O'Reilly Media.

Martin, Robert C. 2018. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston: Prentice Hall.

Meszaros, Gerard. 2007. *xUnit Test Patterns: Refactoring Test Code*. Upper Saddle River, NJ: Addison-Wesley.

Gamma, Erich, Richard Helm, Ralph Johnson, and John Vlissides. 1994. *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.